

# ZeJaymes — Staging vs Migration Latency Comparison

Execution timing, broker send path, and open SL-drift failures for user `zejaymes`

|          |                        |            |                                                                                    |
|----------|------------------------|------------|------------------------------------------------------------------------------------|
| SEVERITY | Info / Open follow-up  | STATUS     | Latency OKSL drift fix pending deploy                                              |
| DATE     | 2026-09-24             | COMPONENT  | Trade worker pipeline (staging + migration)                                        |
| USER     | zejaymes (James Peter) | ROOT CAUSE | No latency regression; V1 reconcile ghost-leg loop (fixed in code, pending deploy) |

## 1. Executive summary (plain English)

We compared how fast the same user’s trades open on the staging environment versus the migration environment. Entry speed is essentially the same on both — about 5 to 6 seconds from the moment a signal is received until the trade appears filled. Migration is not slower; on the raw broker HTTP send it is slightly faster. Separately, migration repeatedly tried (and failed) to fix a stop-loss that did not match the broker. The real problem was that the background “fix the stop” job kept retrying a trade the broker no longer listed as open, instead of closing our leftover open record. That is a sync/cleanup bug on the MTAPI path, not a copy-speed problem; the code fix is written and awaiting deploy.

## 2. Accounts compared

| Environment | Project              | User ID                              | Profile created |
|-------------|----------------------|--------------------------------------|-----------------|
| Staging     | axdcledcyhyvzrnfkwat | f1d54bc2-d6d4-4c4e-be94-13b11bc7d049 | 2026-07-23      |
| Migration   | supmsgcubipmmowrzoub | 20733d28-c337-4dfd-b985-ada9b3609635 | 2026-08-19      |

Same username `zejaymes`, display name James Peter. Sample window: recent trades and `trade_execution_logs` as of 2026-09-24 ~09:10 UTC.

## 3. Recent trades (both envs)

| Time (UTC) | Symbol  | Side | Lot  | Staging ticket | Migration ticket |
|------------|---------|------|------|----------------|------------------|
| 09:07:44   | XAUUSDm | sell | 0.01 | 3275911511     | 3275911509       |
| 08:52:21   | USDJPYm | buy  | 0.01 | 3275802624     | 3275802668       |
| 08:52:08   | EURUSDm | sell | 0.01 | 3275801985     | 3275801962       |
| 08:42:55   | GBPUSDm | sell | 0.01 | 3275755535     | 3275755570       |
| 08:35:48   | XAUUSDm | sell | 0.01 | 3275713604     | 3275713716       |

Same smoke-test set on both environments; broker tickets filled on both MTAPI demo sessions.

## 4. End-to-end latency (signal → trade row)

| Metric      | Staging  | Migration | Notes                                  |
|-------------|----------|-----------|----------------------------------------|
| Samples (n) | 11       | 12        | Linked trades with signal timestamps   |
| p50         | 5,010 ms | 6,236 ms  | Migration +1.2 s median on this sample |
| avg         | 5,533 ms | 5,651 ms  | Within ~100 ms                         |
| p90         | 7,946 ms | 7,129 ms  | Migration better at p90                |
| max         | 8,146 ms | 7,956 ms  | Similar ceiling                        |

| Paired signal | Staging  | Migration | Faster    |
|---------------|----------|-----------|-----------|
| XAUUSD 09:07  | 7,946 ms | 6,515 ms  | Migration |
| USDJPY 08:52  | 4,516 ms | 4,802 ms  | Staging   |
| EURUSD 08:52  | 5,329 ms | 4,852 ms  | Migration |
| GBPUSD 08:42  | 4,428 ms | 4,797 ms  | Staging   |

No consistent winner per symbol. Overall average is effectively the same; migration is not introducing a copy-speed regression.

## 5. Pipeline breakdown ( pipeline\_summary )

| Field             | Staging p50 | Migration p50 | Comment                                 |
|-------------------|-------------|---------------|-----------------------------------------|
| queue_wait_ms     | 262 ms      | 246 ms        | Similar                                 |
| parse_ms          | 114 ms      | 136 ms        | Similar                                 |
| handle_ms         | 5,604 ms    | 879 ms        | Migration lower (sample n=15)           |
| broker_resolve_ms | 223 ms      | 291 ms        | Staging slightly better                 |
| broker_send_ms    | 443 ms      | 306 ms        | MTAPI HTTP send faster on migration     |
| order_send_ms     | 5,005 ms    | 4,545 ms      | Migration slightly better               |
| total_ms          | 8,108 ms    | 7,183 ms*     | *Some migration samples have total_ms=0 |

Staging sample sizes are larger (n=73–85) than migration (n=5–15). Treat migration percentiles as directional.

## 6. Errors and open issue (migration only)

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

| Action   status                    | Staging | Migration |
|------------------------------------|---------|-----------|
| pipeline_summary   success         | 85      | 15        |
| dispatch_push_attempt   success    | 146     | 11        |
| dispatch_push_attempt   failed     | 0       | 3         |
| basket_reconcile_tick   failed     | 0       | 460       |
| merge_routed_modify_only   skipped | 31      | 1         |

**Root cause found — migration SL-drift loop (460 failed ticks, 04:07–09:07 UTC):** On the MTAPI path the V1 basket reconcile monitor kept retrying two XAUUSD baskets ( `1fcc2ff` and `5260104c` ) after the broker ticket was missing from `/OpenedOrders` . Legs were skipped as `skipped_not_on_broker` (not a hard broker error), but the post-pass still compared the database stop against the freshly resolved channel target and logged `Reconcile: 0/1 legs; broker SL still drifted every ~70 seconds. An empty OpenedOrders snapshot (session flake) was treated as “nothing open,”` which forced every leg down that path. Unlike the merge-refresh and V2 loops, the V1 monitor never closed ghost legs and never treated an empty snapshot as inconclusive — so it retried an impossible modify until the trade was closed elsewhere ( `1fcc2ff` closed 09:07:42) or a later snapshot recovered ( `5260104c` success 08:28). Fix: empty snapshot → release job as inconclusive; all legs missing from a non-empty snapshot → `closeStaleOpenTrades` + mark job done; failure message now includes “N not on broker”. See scratchpad `docs/scratchpad-zejaymes-migration-sl-drift-2026-09-24.md` .

## 7. Verification

- REST queries via service role on both projects: `trades` , `signals` , `trade_execution_logs` , `user_profiles` .
- End-to-end latency = `trades.created_at - signals.created_at` for linked `signal_id` , window 0–10 minutes.
- Pipeline metrics from `request_payload` on `action=pipeline_summary` .
- Source: live DB on 2026-09-24; not a synthetic load test.

## 8. Deployment / environment state

- Staging: FxSocket-era worker path; Zejaymes has longer trade history.
- Migration: MTAPI bridge path; token auth cutover complete same day; Zejaymes has fewer historical trades (newer account).
- No code change required for latency based on this sample.

## 9. Follow-ups

1. **Deploy the V1 reconcile guard** — empty OpenedOrders → inconclusive release; all legs missing from a non-empty snapshot → `closeStaleOpenTrades` + mark job done ( `worker/src/basketSLTpReconcileMonitor.ts` ).
2. Optional: re-run comparison after more migration samples to firm up percentiles.
3. Monitor migration `dispatch_push_attempt` failures (3 in window).